Focusly 番茄时钟项目 claudecode 开发 Prompt

Prompt 1：项目分析与开发规划

你现在是这个项目的高级前端工程师，请负责完成一个名为 Focusly——番茄时钟 • 专注学习打卡工具的前端项目。
我已经提供了项目需求说明书，请先完整阅读需求说明书，再开始工作。
第一阶段只进行分析，不要急着写业务代码。
请完成以下工作：
1. 完整理解项目需求，明确项目的项目定位、用户场景、四大核心功能模块、非功能需求、数据结构、API 接口规范、技术栈约束以及最终交付标准。
2. 分析项目应该采用怎样的前端工程架构。
3. 给出推荐的项目目录结构，例如 components、views、composables、services/api、utils、stores、assets 等，并说明每一层的职责。
4. 明确四大功能模块之间的数据关系：
番茄计时、学习任务、每日打卡、数据统计。
5. 重点设计数据层：
Axios 请求层、Mock API、LocalStorage、API 失败后的本地兜底、数据同步策略。
6. 重点分析番茄钟的计时器设计：
如何避免 setInterval 重复创建、如何避免计时漂移、暂停/继续/重置如何处理、页面切换如何清理定时器、学习结束如何进入休息状态、休息结束如何恢复学习状态。
7. 给出页面结构设计。
8. 给出组件拆分方案。
9. 给出开发顺序，并说明每一步的验收标准。
10. 指出需求说明书中可能存在的歧义、潜在 Bug 或者需要在实现时特别注意的地方。
重要约束：
当前阶段不要创建大量业务代码。
不要擅自增加需求说明书没有要求的大型功能。
不要把项目改成 React。
技术栈必须以 Vue3 + Composition API + Axios + ECharts 为核心。
Mock API + LocalStorage 双层数据方案必须保留。
不要使用 TypeScript，除非现有项目已经是 TypeScript。
如果项目目录已经存在，请先检查现有代码，不要直接覆盖。
后续所有代码都必须建立在本阶段确定的架构之上。
最后请输出：
A. 你的需求理解
B. 技术架构
C. 目录结构
D. 数据流设计
E. 组件设计

F. 开发计划
G. 风险点
暂时不要开始大规模编码。

Prompt 2：创建工程与基础架构

现在开始第二阶段。
请根据上一阶段确定的架构方案，正式开始搭建 Focusly 项目。
目标是先完成一个可以正常启动运行的 Vue3 前端工程，不实现全部业务功能。
技术要求：
Vue 3、Composition API、Vite、JavaScript、Axios、ECharts、CSS3、LocalStorage。
不使用 TypeScript，不引入不必要的大型 UI 框架。
本阶段完成以下工作：
1. 创建或完善项目基础工程。
2. 配置 package.json 和 Vite。
3. 建立合理的目录结构。
4. 创建基础页面和组件。
建议至少包含：
src/
assets/
components/
views/
composables/
services/
api/
utils/
router/
stores/
App.vue
main.js
如果根据实际项目情况需要调整目录，可以调整，但必须说明原因。
5. 建立基础 API 请求封装：
Axios 实例、baseURL、timeout、请求拦截器、响应拦截器、统一错误处理。
6. 建立 LocalStorage 工具类。
7. 建立统一的数据结构约定。
8. 创建基础布局：
顶部区域、核心计时区域、学习任务区域、打卡区域、数据统计区域。
9. 完成基础 CSS 和响应式布局。
10. 确保执行 npm install 后可以通过 npm run dev 正常启动。
重要要求：
这一阶段重点是工程架构，不要把所有业务逻辑塞进 App.vue。
组件必须保持单一职责。
完成后：

1. 检查所有 import 路径。
2. 检查控制台是否有报错。
3. 检查 npm run dev 是否正常。
4. 给出本阶段修改的文件列表。
5. 简述下一阶段应该实现什么。

不要擅自实现需求说明书之外的功能。

Prompt 3：页面 UI 与响应式布局

现在开始第三阶段：完成 Focusly 的页面 UI 和响应式布局。
本阶段暂时不重点处理复杂的数据持久化和 API 联调，重点把页面结构、视觉效果和交互基础做好。
设计目标：
整体风格为极简、清新、学生学习工具、专注感、无广告、不花哨、信息层级清晰。
严格按照需求说明书的产品定位设计。
页面需要包含以下区域：
1. Header：
Focusly 品牌名称、简短副标题或说明，保持简洁。
2. 番茄钟核心区域：
当前模式：专注/休息；
大型倒计时；
开始；
暂停；
重置；
学习时长设置；
休息时长设置。
3. 学习任务区域：
新增任务；
任务列表；
完成状态；
编辑；
删除。
4. 每日打卡区域：
今日学习状态；
打卡按钮；
最近打卡记录。
5. 数据统计区域：
今日专注时间；
近 7 天统计；
近 30 天统计；
为 ECharts 预留图表区域。
UI 要求：

使用 CSS 实现，不要依赖大型 UI 组件库。
采用卡片布局。
圆角适中。
阴影轻量。
按钮具有 hover 和 active 反馈。
完成任务后有明显但不过度的视觉变化。
保持页面简洁。
响应式要求：
必须适配 1920px 桌面、1440px 桌面、1024px 平板、768px 以及 390px 手机。
手机端不允许出现横向滚动，不允许组件被挤压。
计时器仍然是视觉中心。
任务、打卡、统计区域自动纵向排列。
代码要求：
App.vue 不要承担所有 UI 代码。
合理拆分组件。
CSS 命名清晰。
不要写大量重复 CSS。
不要使用内联 style 堆砌页面。
完成后运行项目检查：
页面是否正常显示；
是否存在控制台错误；
是否存在横向滚动；
手机端布局是否正常。
本阶段重点是把页面做好看、结构做好，业务逻辑下一阶段再深入实现。

Prompt 4：番茄钟核心功能

现在开始第四阶段，只实现并完善番茄专注计时模块。
请不要在这一阶段扩展到其他复杂功能。
功能要求：
1. 默认专注 25 分钟，休息 5 分钟。
2. 支持用户自定义专注时长和休息时长。
3. 输入校验：
空值禁止；
非数字禁止；
小于等于 0 禁止；
非法输入给出友好提示。
4. 计时控制：
开始；
暂停；
继续；
重置。
5. 显示：

MM:SS；
如果设计需要，也可以显示 HH:MM:SS。
6. 模式：
focus；
rest。
7. 专注结束：
自动切换到休息模式；
提示用户进入休息；
重新加载休息倒计时。
8. 休息结束：
提示用户休息结束；
可以继续进入专注。
最重要的是计时器稳定性。
必须避免：
setInterval 重复创建；
点击开始多次导致多个 timer 同时运行；
暂停后仍然计时；
重置后旧 timer 继续运行；
页面卸载后 timer 仍然运行；
长时间运行出现明显计时漂移；
切换组件后出现幽灵 timer；
内存泄漏。
请优先考虑基于时间戳计算剩余时间，而不是简单地依赖“每秒减 1”。
例如：
endTime = Date.now() + remainingSeconds * 1000
每次 tick 根据：
remaining = endTime - Date.now()
重新计算剩余时间。
这样可以降低 setInterval 导致的时间漂移。
状态设计：
请明确区分当前模式、当前剩余时间、初始时间、是否正在运行、是否暂停、是否完成。
避免使用互相冲突的多个 boolean。
学习记录：
每完成一次完整的专注周期，需要产生一条专注学习记录，为后续打卡和统计提供数据。
请把这部分逻辑设计成独立的数据处理函数，不要直接写死在 UI 组件中。
生命周期：
组件卸载时必须清理 timer。
验收测试：
1. 连续点击开始 10 次。
2. 开始→暂停→继续。
3. 开始→重置。
4. 重置→开始。

5. 专注结束。
6. 休息结束。
7. 修改时间后开始。
8. 页面刷新。
9. 切换页面或组件。
10. 长时间运行。
完成后检查控制台、timer 数量和状态切换，并给出核心实现说明和修改文件列表。
不要为了方便而降低上述稳定性要求。

Prompt 5：学习任务管理

现在开始第五阶段，实现学习任务清单模块。
严格按照需求说明书实现，不增加复杂的任务系统。
数据结构：
{
id,
content,
status,
createTime
}
status：
0 = 未完成；
1 = 已完成。
功能：
1. 新增任务：
输入任务内容；
禁止空任务；
添加后立即显示。
2. 修改任务：
支持编辑任务内容；
保存后更新列表。
3. 完成/取消完成：
点击切换 status；
已完成任务文字置灰；
添加删除线。
4. 删除单个任务。
5. 清空所有任务。
6. 数据持久化：
页面刷新后任务不能丢失。
数据层要求：
不要直接在组件中到处操作 localStorage。
应该通过统一的数据服务层处理。

例如：
taskApi.getList()
taskApi.add()
taskApi.update()
taskApi.delete()
如果 Mock API 可用，优先请求 API。
如果 API 失败，自动使用 LocalStorage。
API 成功后，同步 LocalStorage。
交互：
添加成功提示；
修改成功提示；
删除成功提示；
非法输入提示；
请求 loading 状态。
代码要求：
合理拆分任务列表组件、任务项组件、任务输入组件、API/service 层以及 LocalStorage 兜底。
完成后测试：
添加、修改、完成、取消完成、删除、清空、刷新页面、API 失败、LocalStorage 恢复。
不要修改已经稳定的番茄钟核心逻辑，除非任务模块与计时记录联动确实需要。

Prompt 6：每日打卡与数据统计

现在开始第六阶段，实现：
1. 每日学习打卡。
2. 数据统计。
3. ECharts 可视化。
一、每日打卡
数据结构：
{
date: "YYYY-MM-DD",
studyTime: 分钟,
createTime: 时间
}
功能：
1. 今日打卡。
2. 每天只能打卡一次。
3. 重复打卡时给出明确提示。
4. 保存当天累计专注时长。
5. 展示近期打卡记录。
6. 已打卡日期高亮。
特别注意：

日期判断必须统一使用本地日期格式 YYYY-MM-DD。
不要简单使用 toISOString()导致时区问题。
例如日本、中国等 UTC+8/UTC+9 地区，凌晨附近可能出现日期错位。
请封装统一的 date 工具。
二、专注时长统计
基于已有专注记录计算：
今日累计；
最近 7 天；
最近 30 天。
统计数据必须动态计算。
用户新增专注记录后，统计自动更新。
三、ECharts
使用 ECharts。
近 7 天使用柱状图或者折线图，X 轴为日期，Y 轴为分钟。
近 30 天使用折线图优先，展示学习时长趋势。
要求：
图表响应式；
窗口改变自动 resize；
组件卸载时释放 ECharts 实例；
数据变化后自动更新；
没有数据时显示友好的 empty 状态。
四、统计卡片
至少展示：
今日专注时长；
近 7 天专注时长；
近 30 天专注时长；
今日是否打卡。
五、数据一致性
必须保证：
番茄钟产生专注记录 → 学习记录数据 → 打卡 → 统计 → ECharts。
这一数据链路能够正常工作。
不要为了实现统计而重新建立一套互相独立的数据。
完成后进行完整测试：
没有专注记录；
有 1 条记录；
有多条记录；
跨日期；
跨月；
今日打卡；
重复打卡；
刷新页面；
图表 resize；
手机端图表。
最后列出所有修改文件以及数据流。

Prompt 7：Mock API 与 LocalStorage 双层数据架构

现在开始第七阶段，对整个项目的数据层进行工程化完善。
目标：
实现：
前端业务 → Service/API 层 → Axios → Apifox Mock API。
API 失败 → LocalStorage 兜底。
API 成功 → 同步 LocalStorage。
API 必须按照需求说明书。
基础地址：
/api
接口：
GET /api/timer/config
PUT /api/timer/config
GET /api/task/list
POST /api/task/add
PUT /api/task/update
DELETE /api/task/delete
GET /api/clock/list
POST /api/clock/add
GET /api/stat/week
GET /api/stat/month
统一响应：
{
code: 200,
msg: "",
data: {}
}
架构要求：
禁止在 Vue 组件中直接调用 axios。
统一采用：
Component → Composable → Service → Axios → Mock API。
例如：
taskService.getTasks()
taskService.addTask()
taskService.updateTask()
taskService.deleteTask()
timerService.getConfig()
timerService.saveConfig()
clockService.getRecords()
clockService.addClock()
statService.getWeekStats()
statService.getMonthStats()

LocalStorage 统一封装：
storage.get()
storage.set()
storage.remove()
统一 key 命名。
不要在多个组件中出现：
localStorage.setItem(...)
这种散乱代码。
失败兜底：
如果 API 超时、API 返回错误、网络错误或者 Mock 服务不可用，自动切换 LocalStorage。
同时：
给用户合理提示；
不影响页面继续使用。
加载状态：
API 请求期间显示 loading。
避免重复请求。
避免同一个接口短时间内重复提交。
数据一致性：
API 成功：
API → LocalStorage。
API 失败：
LocalStorage → UI。
如果设计上支持恢复同步：
LocalStorage → API。
请避免出现两套数据源互相覆盖的问题。
完成后模拟以下场景：
1. API 正常。
2. API 失败。
3. 网络断开。
4. LocalStorage 没有数据。
5. LocalStorage 已有数据。
6. API 成功后刷新页面。
7. API 失败后刷新页面。
最后说明完整的数据流和异常处理策略。

Prompt 8：全面测试与 Bug 修复

现在不要新增功能。
请把自己当成这个项目的高级测试工程师和代码审查工程师，对整个 Focusly 项目进行全面测试。
目标是：发现问题 → 定位问题 → 修复问题 → 回归测试。
一、功能测试

请逐项测试：
番茄钟：
开始、暂停、继续、重置、自定义时间、非法输入、专注结束、休息结束、连续点击开始、页面刷新。
任务：
添加、修改、完成、取消完成、删除、清空、空任务。
打卡：
今日打卡、重复打卡、历史打卡、日期变化。
统计：
今日、7 天、30 天、无数据、多条数据。
二、数据测试
测试：
API 成功；
API 失败；
API 超时；
LocalStorage 已有数据；
LocalStorage 没有数据；
刷新页面；
关闭浏览器重新打开。
检查数据是否丢失。
三、计时器专项测试
重点检查：
是否存在多个 setInterval；
是否存在 timer 泄漏；
暂停后是否真的停止；
重置后旧 timer 是否还运行；
组件卸载是否清理；
长时间运行是否产生明显漂移；
专注→休息状态是否正确。
四、响应式测试
至少检查：
1920；
1440；
1024；
768；
390。
检查：
横向滚动；
按钮溢出；
文字溢出；
图表溢出；
卡片挤压；
计时器布局。
五、代码审查

检查：
是否存在重复代码；
是否存在无用代码；
是否存在 console.log；
是否存在硬编码；
是否存在巨型组件；
API 是否全部经过 service 层；
LocalStorage 是否统一封装；
是否存在潜在内存泄漏；
是否存在 Promise 未处理；
是否存在异常未捕获。
六、修复原则
发现问题后直接修复。
不要为了“看起来没问题”而跳过问题。
修复后必须重新测试相关功能。
最后输出：
1. 发现的问题。
2. 问题原因。
3. 修复方案。
4. 修改文件。
5. 测试结果。
6. 仍然存在的风险。
不要新增需求。

Prompt 9：UI 与响应式体验优化

现在项目核心功能已经完成。
这一阶段只进行 UI、交互和响应式体验优化，不改变核心业务逻辑。
整体目标：
把当前项目从“功能能运行”提升到“课程实训成品”。
风格：
极简、清爽、学习工具、专业、有一定产品感，不要过度设计。
重点优化：
1. 页面整体视觉层次。
2. Header。
3. 番茄钟作为页面视觉中心。
4. 开始/暂停/重置按钮。
5. 时间设置区域。
6. 任务卡片。
7. 打卡区域。
8. 统计卡片。
9. ECharts 图表。
10. 空数据状态。

11. Loading 状态。
12. Error 提示。
13. Success 提示。
14. Hover 动画。
15. 点击反馈。
响应式：
重点检查 390px 手机。
要求：
无横向滚动；
计时器不变形；
按钮不挤压；
输入框不溢出；
ECharts 不溢出；
卡片自动排列；
文字正常换行。
动画：
可以增加 hover、button active、card hover、页面进入等效果，但动画必须克制。
不要加入复杂动画库。
重要：
不要修改 API 结构、数据结构、番茄钟核心计时算法、Service 层架构。
如果 UI 问题必须修改组件结构，可以修改，但要保证功能不受影响。
完成后重新运行项目并检查控制台。

聊天记录导出

This session is being continued from a previous conversation that ran out of context. The summary below covers the earlier portion of the conversation.
Summary:
1. Primary Request and Intent:
The user is building **"Focusly——番茄时钟 • 专注学习打卡工具"** (a Pomodoro-focused study/check-in tool) as a front-end training project, progressing through explicit numbered phases. Working directory: `d:\番茄钟\番茄钟代码`. Requirements docs are in `d:\番茄钟\Hands-on-Training-Materials-main\`.
Hard constraints stated repeatedly across phases (MUST be preserved verbatim in effect):
- Tech stack MUST be: Vue3 (Composition API) + Vite + JavaScript + Axios + ECharts + CSS3 + LocalStorage
- Mock API + LocalStorage 双层数据方案 must be preserved (Mock API as primary channel, LocalStorage as offline fallback)
- Do NOT use TypeScript, do NOT use React, do NOT introduce large UI component libraries
- Do NOT add features beyond the requirements spec
- Check existing code before overwriting

- Do NOT modify the already-stable Pomodoro core timer logic (stated in Phase 5 and reaffirmed)
Phase 1 (complete): Analysis only, sections A-G, no business code.
Phase 2 (complete): Runnable Vue3 project foundation.
Phase 3 (complete): UI and responsive layout polish.
Phase 4 (complete): Pomodoro timer module only (defaults 25/5, custom durations, validation, start/pause/resume/reset, MM:SS, focus/rest modes, timestamp-based countdown, single status enum, learning record as independent function, lifecycle cleanup, 10 acceptance tests).
Phase 5 (complete): Task management + daily check-in + stats/ECharts visualization, with unified data chain (timer → focus record → check-in → stats → ECharts), no duplicate data sources.
Phase 7 (CURRENT — NOT yet started): Data-layer engineering refinement with these exact requirements:
- Data flow: 前端业务 → Service/API 层 → Axios → Apifox Mock API; API 失败 → LocalStorage 兜底; API 成功 → 同步 LocalStorage
- Base address: `/api`
- API endpoints (MUST follow requirements spec):
  - GET/PUT `/api/timer/config`
  - GET `/api/task/list`, POST `/api/task/add`, PUT `/api/task/update`, DELETE `/api/task/delete`
  - GET `/api/clock/list`, POST `/api/clock/add`
  - GET `/api/stat/week`, GET `/api/stat/month`
- Unified response: `{code:200, msg:"", data:{}}`
- Architecture: Component → Composable → Service → Axios → Mock API; 禁止在 Vue 组件中直接调用 axios
- Service naming examples: `taskService.getTasks()/addTask()/updateTask()/deleteTask()`, `timerService.getConfig()/saveConfig()`, `clockService.getRecords()/addClock()`, `statService.getWeekStats()/getMonthStats()`
- LocalStorage unified wrapper: `storage.get()`, `storage.set()`, `storage.remove()`, unified key naming, no scattered `localStorage.setItem(...)` in components
- Failure fallback (API timeout/error/network/mock unavailable → LocalStorage, with user-friendly prompt, page still usable)
- Loading states during API requests; avoid duplicate requests; avoid repeated submission of same interface in short time
- Data consistency: API 成功 → LocalStorage; API 失败 → LocalStorage → UI; if recovery sync supported → LocalStorage → API; avoid two data sources overwriting each other
- After completion, simulate 7 scenarios: (1) API normal, (2) API failure, (3) network offline, (4) LocalStorage empty, (5) LocalStorage has data, (6) refresh after API success, (7) refresh after API failure
- Finally explain complete data flow and exception handling strategy
2. Key Technical Concepts:

- Vue 3 Composition API, `<script setup>`, module-level `reactive()` singletons (composable store pattern, NO Pinia)
- Vite 5.4, `@` alias → `./src`, `import.meta.env.VITE_API_BASE_URL`/`VITE_API_TIMEOUT`
- vue-router 4, `createWebHashHistory` (single HomeView + catch-all redirect)
- Axios instance with interceptor: request (loading counter), response (unwrap `{code,msg,data}`, strict `typeof res.code === 'number' && res.code === 200`)
- Mock + LocalStorage dual-layer: `readWithFallback` (API-first → local fallback), `writeWithFallback` (optimistic local + pending sync queue), `replayPendingSync`
- **Local mode short-circuit**: `const isLocalMode = !import.meta.env.VITE_API_BASE_URL` — when baseURL empty, adapter skips API and reads/writes LocalStorage directly (avoids failed requests + console warnings)
- Timestamp-based countdown: `endAt = now() + remaining*1000`, `remaining = Math.ceil((endAt - now())/1000)`
- Single `status` enum (idle|running|paused) + `mode` enum (focus|rest); "completed" as transient `onComplete` callback
- Pure-logic extraction for Node testability: `timerCore.js` (injected now/setInterval/clearInterval), `stats.js` `aggregateDaily`, relative imports WITH `.js` extension for Node ESM
- ECharts full import, init/resize/dispose lifecycle, `watch(() => [props.data, props.range], render)`
- CSS design tokens (variables.css), responsive breakpoints (768px, 480px)
- Daily focus log pattern: `log = [{date:'YYYY-MM-DD', minutes}]` as single source of truth for duration

3. Files and Code Sections:

- `src/constants/index.js` (EDITED Phase 4 + 5)
  - Central constants. Phase 5 renamed `TODAY_STUDY` → `STUDY_LOG`.
  - STORAGE_KEYS: `{ TIMER_CONFIG:'focusly:timerConfig', TASKS:'focusly:tasks', CLOCKS:'focusly:clocks', STUDY_LOG:'focusly:studyLog', PENDING_SYNC:'focusly:pendingSync' }`
  - `TIMER_DEFAULT = { focusDuration: 25, restDuration: 5 }`, `TIMER_RANGE = { focus:{min:1,max:180}, rest:{min:1,max:60} }`, `TIMER_MODE = {FOCUS:'focus', REST:'rest'}`, `TIMER_STATUS = {IDLE:'idle', RUNNING:'running', PAUSED:'paused'}`, `TASK_STATUS = {TODO:'0', DONE:'1'}`, `TASK_LIMIT = {contentMax:60, descriptionMax:160}`, `STAT_RANGE = {WEEK:'week', MONTH:'month'}`
- `src/composables/timerCore.js` (Phase 4, stable, NOT modified in Phase 5/7)
  - Pure state machine. `createTimerCore({state, now, setIntervalFn, clearIntervalFn, tickMs=250})`. Single `timerId`, `stopTick()` as sole clear entry, `beginTick()` stops then starts, `complete()` switches mode + reloads countdown + `onComplete(finished)`. `dispose()` clears timer, running→paused.
- `src/composables/useTimer.js` (Phase 4, stable)
  - Vue shell. Loads config from LocalStorage (`focusDuration`/`restDuration`).

Session snapshots `sessionTotalSeconds`/`focusSessionMinutes` captured at `start()`. `onComplete` → `recordFocusSession(focusSessionMinutes)` + `show('专注结束，休息一下吧')`/`show('休息结束，继续专注吧')`. `visibilitychange` handler calls `core.syncRemaining()`. Exposes `dispose`.
- `src/composables/useStudyRecord.js` (REWRITTEN Phase 5)
  - Daily focus log as single source of truth:
```js
const state = reactive({ todayMinutes: 0, log: [] })
function load() { const saved = getStorage(STORAGE_KEYS.STUDY_LOG, []); state.log = Array.isArray(saved) ? saved : [] }
function syncToday() { const today = formatDate(); const entry = state.log.find((e) => e.date === today); state.todayMinutes = entry ? entry.minutes : 0 }
function persist() { setStorage(STORAGE_KEYS.STUDY_LOG, state.log) }
export function recordFocusSession(minutes) { /* find/create today entry, entry.minutes += minutes, state.todayMinutes = entry.minutes, persist() */ }
load(); syncToday()
export default function useStudyRecord() { return { state, recordFocusSession } }
```
- `src/composables/useClock.js` (REWRITTEN Phase 5)
  - `state = {records:[], todayClocked:false}`. `syncTodayClocked()` sets todayClocked from records. `loadClocks()` via `readWithFallback(getClockList, STORAGE_KEYS.CLOCKS, [])`. `clockIn()` guards `if (state.todayClocked) { show('今天已经打过卡啦'); return false }`, creates `{date, studyTime: studyState.todayMinutes, createTime: Date.now()}`, pushes, `writeWithFallback(STORAGE_KEYS.CLOCKS, state.records, {url:'/clock/add', method:'post', data:record})`, `show('打卡成功')`. Calls `loadClocks()` at module init.
- `src/composables/useTask.js` (REWRITTEN Phase 5)
  - `state = {tasks:[], loading:false}`. `genId()` = `'task_' + Date.now().toString(36) + '_' + Math.random().toString(36).slice(2,6)`. `addTask({content, description=''})` (rejects empty), `toggleTask(id)`, `updateTask({id, content})`, `removeTask(id)`, `clearTasks()` (uses `setStorage(STORAGE_KEYS.TASKS, [])` directly). Writes via `persist(syncConfig)` = `writeWithFallback(STORAGE_KEYS.TASKS, state.tasks, syncConfig)`. Calls `loadTasks()` at module init.
- `src/composables/useStats.js` (REWRITTEN Phase 5)
  - Imports useStudyRecord + useClock. `summary = computed(() => ({today, week: aggregateDaily(log,7).total, month: aggregateDaily(log,30).total}))`, `trend = computed(() => aggregateDaily(log, range===='month'?30:7).trend)`, `todayClocked = computed(() => clockState.todayClocked)`. Returns `{state, summary, trend, todayClocked, setRange}`.
- `src/utils/stats.js` (NEW Phase 5)
```js
import { getRecentDates } from './date.js'
export function aggregateDaily(log, days, end = new Date()) {
  const dates = getRecentDates(days, end)
```

```
  const byDate = new Map((log || []).map((e) => [e.date, e.minutes]))
  let total = 0
  const trend = dates.map((date) => { const studyTime = byDate.get(date) ?? 0; total += studyTime; return { date, studyTime } })
  return { total, trend }
}
```

- `src/utils/storage.js` (EXISTING, NOT renamed — NOTE: Phase 7 wants `storage.get()/set()/remove()` naming but current is `getStorage`/`setStorage`/`removeStorage`/`clearStorageByPrefix`)
  - `getStorage(key, fallback=null)`, `setStorage(key, value)`, `removeStorage(key)`, `clearStorageByPrefix(prefix='focusly:')` — all JSON serialize/parse with try/catch + console.warn.
- `src/utils/validators.js` (EDITED Phase 4)
  - `validateDuration(raw, type)` with `type` 'focus'|'rest', labels '专注时长'/'休息时长', checks empty/non-number/<=0/range; relative import `'../constants/index.js'`. `validateTaskContent(content)`.
- `src/utils/date.js` (EXISTING) — `formatDate(date=new Date())` local YYYY-MM-DD, `getRecentDates(days, end=new Date())` old → new, `getWeekday(dateStr)`.
- `src/utils/format.js` (EXISTING) — `formatSeconds(totalSeconds)` → HH:MM:SS or MM:SS.
- `src/services/request.js` (EXISTING) — axios instance, baseURL `import.meta.env.VITE_API_BASE_URL || ''`, timeout `Number(VITE_API_TIMEOUT)||3000`, loading counter with `onLoadingChange(fn)`, response interceptor unwraps data on `typeof res.code === 'number' && res.code === 200`, else rejects with `res.msg`. Exports default `request(config)`.
- `src/services/mock/adapter.js` (EDITED Phase 5 — added isLocalMode short-circuit)
  - Key for Phase 7. Currently: `const isLocalMode = !import.meta.env.VITE_API_BASE_URL`. `readWithFallback(apiFn, storageKey, fallback)` returns `getStorage(storageKey, fallback)` if local; `writeWithFallback(storageKey, newData, syncConfig)` does `setStorage` then returns if local. In Mock mode: read does `await apiFn()` then `setStorage(storageKey, data)`; write does setStorage + push `syncConfig` to PENDING_SYNC + `request(syncConfig)`. `replayPendingSync()` replays queue.
- `src/services/api/timer.js` — `getTimerConfig()` GET /timer/config, `saveTimerConfig(data)` PUT /timer/config
- `src/services/api/task.js` — `getTaskList()` GET /task/list, `addTask(data)` POST /task/add, `updateTask(data)` PUT /task/update, `deleteTask(id)` DELETE /task/delete (params `{id}`)
- `src/services/api/clock.js` — `getClockList()` GET /clock/list, `addClock(data)` POST /clock/add

- `src/services/api/stat.js` — `getWeekStats()` GET /stat/week, `getMonthStats()` GET /stat/month
  - NOTE: These all use paths WITHOUT `/api` prefix (e.g. `/task/list`), while Phase 7 requires `/api/task/list`. Also file names are `task.js` etc. not `taskService.js`.
- `.env` — `VITE_API_BASE_URL=` (empty → local mode), `VITE_API_TIMEOUT=3000`. `.env.example` documents Mock mode example `VITE_API_BASE_URL=https://m1.apifoxmock.com/m1/1234567-0-default/api`.
- Components (all Phase 5 rewritten unless noted):
  - `src/components/timer/TimerDial.vue` (Phase 4) — props remaining/total/mode/status/modeLabel, `dialClass` computed (rest→mint, idle→gray, else default tomato).
  - `src/components/timer/TimerControls.vue` (Phase 4) — status-driven start/pause/resume/reset buttons.
  - `src/components/timer/DurationInput.vue` (Phase 4) — local draft refs + blur validation via validateDuration, emits `{focus, rest}`.
  - `src/components/timer/TimerPanel.vue` (Phase 4) — wires useTimer, `onBeforeUnmount(() => dispose())`.
  - `src/components/common/AppToast.vue` (Phase 4) — reads useToast singleton state; fixed `--radius-btn`→`--radius-md`.
  - `src/components/common/HeaderBar.vue`, `LoadingMask.vue`, `ConfirmModal.vue` — stable.
  - `src/components/task/TaskItem.vue` (Phase 5) — inline edit (editing/draft refs, save/cancel, emit 'update' with {id, content}), done state gray + line-through.
  - `src/components/task/TaskList.vue` (Phase 5) — emits toggle/update/delete/clear.
  - `src/components/task/TaskPanel.vue` (Phase 5) — wires useTask.
  - `src/components/clock/ClockPanel.vue` (Phase 4+5) — uses useClock `{state, clockIn}` + useStudyRecord `{state: study}`, displays `study.todayMinutes`.
  - `src/components/clock/ClockButton.vue`, `ClockCalendar.vue` — stable.
  - `src/components/chart/StatsPanel.vue` (Phase 5) — consumes `{state, summary, trend, todayClocked, setRange}`.
  - `src/components/chart/StatsSummary.vue` (Phase 5) — 4 cards (今日专注/近 7 天/近 30 天/今日打卡), grid `repeat(4,1fr)` → `repeat(2,1fr)` mobile.
  - `src/components/chart/TrendChart.vue` (Phase 5) — `hasData = data.some(d => d.studyTime > 0)`, bar for week / line (smooth + areaStyle) for month, resize/dispose lifecycle.
- `scripts/verify-timer.mjs` (Phase 4) — 14 tests for timer core + validateDuration, all pass.
- `scripts/verify-stats.mjs` (Phase 5) — 4 tests for aggregateDaily (无记录/多记录 7 天/30 天窗口/同日去重), all pass.
4. Errors and fixes:
- **stats.js Node ESM import failure**: `import { getRecentDates } from './date'` caused `ERR_MODULE_NOT_FOUND: Cannot find module .../src/utils/date`. Fixed by changing to `./date.js` (Node ESM requires extension; Vite handles both).

- **AppToast undefined CSS variable**: `border-radius: var(--radius-btn)` (no such variable in variables.css) → fixed to `var(--radius-md)`.
- **clearTasks passing null syncConfig**: `writeWithFallback(STORAGE_KEYS.TASKS, [], null)` would call `request(null)` in Mock mode → fixed by importing `setStorage` and calling `setStorage(STORAGE_KEYS.TASKS, [])` directly.
- **Naming inconsistency (Phase 4)**: `studyDuration`/`TIMER_RANGE.study`/"学习时长" vs new `focusDuration`/"专注" → aligned everything to "focus" terminology.
- No user feedback correcting my approach; all fixes were self-detected.
5. Problem Solving:
- Reconciled Phase 1 "no router" with Phase 2 router requirement by using vue-router (hash mode, single route).
- Designed unified data source: `useStudyRecord` daily log is the single source for focus duration; `useClock` (check-in) reads from it; `useStats` aggregates from it + clock records. Avoids duplicate data sources.
- Extended `useStudyRecord` from single-day to daily log to support 7-day/30-day stats.
- Added `isLocalMode` short-circuit to adapter to avoid console warnings in local mode (default `.env` has empty baseURL).
- Made `validators.js`/`stats.js`/`timerCore.js` use relative imports with `.js` extension for Node testability without Vite alias.
6. All user messages:
- (Phase 4 continuation) "继续第四阶段，只实现并完善"番茄专注计时模块"" with full timer requirements (defaults 25/5, custom durations, validation empty/non-number/<=0 with friendly messages, start/pause/resume/reset, MM:SS, focus/rest modes, focus-end auto-switch to rest, rest-end continue, CRITICAL timer stability avoiding setInterval duplication/ghost timers/drift, timestamp-based `endTime = Date.now() + remainingSeconds*1000`, state design distinguishing mode/remaining/initial/running/paused/completed, learning record as independent function, lifecycle cleanup, 10 acceptance tests; "不要为了方便而降低上述稳定性要求").
- (Phase 5) "现在开始第五阶段，在当前已有项目基础上，同时完成：1. 学习任务管理 2. 每日学习打卡 3. 学习数据统计与 ECharts 可视化。请不要修改已经稳定的番茄钟核心计时逻辑" with task data structure `{id, content, status, createTime}`, clock data `{date:"YYYY-MM-DD", studyTime, createTime}`, stats (今日/近 7 天/近 30 天/今日是否打卡), ECharts (近 7 天柱状图/折线图, 近 30 天趋势图, auto-update, resize, dispose on unmount, empty-state friendly prompt), data relationship (番茄钟完成专注→产生专注记录→打卡使用累计专注时长→统计聚合→ECharts, 不要建立互相独立重复的数据源), code requirements (API/数据处理不写在组件中, 通过 Service 层和 Composable, LocalStorage 统一通过已有工具封装, 单一职责, 不引入不必要技术), test scenarios, 检查控制台报错, 列出修改文件.
- (Phase 7, CURRENT) "现在开始第七阶段，对整个项目的数据层进行工程化完

善" — the full data-layer requirements (base address `/api`, all endpoints with `/api` prefix, unified response `{code:200,msg:"",data:{}}`, architecture Component → Composable → Service → Axios → Mock API, 禁止在 Vue 组件中直接调用 axios, service naming taskService/timerService/clockService/statService, LocalStorage unified `storage.get()/set()/remove()`, no scattered localStorage.setItem, failure fallback with friendly prompt, loading states, avoid duplicate requests/repeated submits, data consistency rules, simulate 7 scenarios, explain complete data flow and exception handling).

7. Pending Tasks:

- Phase 7 (data-layer engineering) has NOT been started. It requires:
  - Set base address `/api` (likely via `.env` `VITE_API_BASE_URL=/api` or service path changes)
  - Rename/reorganize service layer to match `taskService.getTasks()` etc. naming (current files are `api/task.js` with `getTaskList()`)
  - Ensure API paths match spec (`/api/task/list` etc. — current paths are `/task/list` WITHOUT `/api` prefix)
  - Rename storage wrapper to `storage.get()/set()/remove()` (current is `getStorage`/`setStorage`/`removeStorage`)
  - Implement failure fallback + friendly user prompt + loading states + duplicate request prevention
  - Verify data consistency (no two data sources overwriting each other)
  - Simulate 7 scenarios and document data flow/exception strategy

8. Current Work:

Immediately before this summary request, I had just COMPLETED Phase 5 (task management, daily check-in, stats/ECharts). The last actions were: running `node scripts/verify-stats.mjs` (4/4 pass) + `node scripts/verify-timer.mjs` (14/14 pass), running `npm run build` (690 modules, passed, ECharts >500kB warning is known/non-blocking), and a final grep confirming no stale references (`TODAY_STUDY`/`todayStudyTime`/`TODO(Phase x)`/`@edit`/`state.phase`/`state.running` all clean). I then marked all Phase 5 todos complete and delivered the Phase 5 completion summary.

The user's most recent message is the Phase 7 request (data-layer engineering). This has NOT been started — the CRITICAL instruction in the current message ("Respond with TEXT ONLY. Do NOT call any tools") is a summarization trigger, not the Phase 7 implementation itself.

9. Optional Next Step:

Begin Phase 7 by first auditing the current data layer against the new requirements, since there are known discrepancies to resolve before writing code:

- Current service paths are `/task/list` etc. (no `/api` prefix); Phase 7 requires `/api/task/list`. Likely fix: set `VITE_API_BASE_URL=/api` in `.env` (base address is explicitly `/api` in the requirement) OR add `/api` to each service path — must confirm which is intended.
- Current storage wrapper uses `getStorage`/`setStorage`/`removeStorage`; Phase 7 requires `storage.get()/set()/remove()` naming.

- Current service files are `api/task.js`/`clock.js`/`stat.js`/`timer.js` with functions `getTaskList()`/`getClockList()`/`getWeekStats()`; Phase 7's examples show `taskService.getTasks()`, `clockService.getRecords()`, `statService.getWeekStats()`, `timerService.getConfig()`.

The most recent user instruction to act on is: "现在开始第七阶段，对整个项目的数据层进行工程化完善" with "【API 必须按照需求说明书】" (base `/api`, endpoints `/api/task/list` etc.) and "LocalStorage... 统一封装： storage.get() / storage.set() / storage.remove()". I should not start writing code until I resolve the `/api` prefix placement (env baseURL vs service paths) and the naming conventions, since these are architectural decisions affecting many files. I will proceed with the Phase 7 implementation after this summary is generated.

If you need specific details from before compaction (like exact code snippets, error messages, or content you generated), read the full transcript at: C:\Users\梁恩收\.claude\projects\d----------\8a74d279-eaa5-444a-b940-c49749368ded.jsonl

Continue the conversation from where it left off without asking the user any further questions. Resume directly — do not acknowledge the summary, do not recap what was happening, do not preface with "I'll continue" or similar. Pick up the last task as if the break never happened.